Ex no: 5(a)	Running an Existing Docker Container and Building a New Image
Date:	

Aim:

The aim of this project is to understand and demonstrate the usage of Docker by running an existing container and building a new Docker image. This includes managing containerized applications, verifying container execution, and creating custom images to optimize application deployment, scalability, and portability.

Docker:

Docker is an open-source containerization platform that automates the process of creating, deploying, and managing containers. It enables developers to package an application along with all its dependencies into a container image, ensuring the application runs consistently across different environments. This eliminates the common problem of **"It works on my machine,"** as the container behaves the same way whether it is running on a laptop, server, or cloud.

Purpose of Docker:

- **Portability** – Run applications anywhere (Linux, Windows, Cloud, On-Premises).
- **Consistency** – Same environment across development, testing, and production.
- **Isolation** – Multiple apps run independently without interfering.
- **Efficiency** – Containers are lightweight and use fewer resources than VMs.
- **Scalability** – Supports scaling with Kubernetes or Docker Swarm.
- **Faster Deployment** – Containers start/stop quickly compared to VMs.

Commands and Output:

docker –version

```
C:\Program Files\Docker\Docker\resources\bin>docker --version
Docker version 29.2.1, build a5c7197
```

docker-info

```
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\Docker\Docker\resources\bin>docker --version
Docker version 29.2.1, build a5c7197

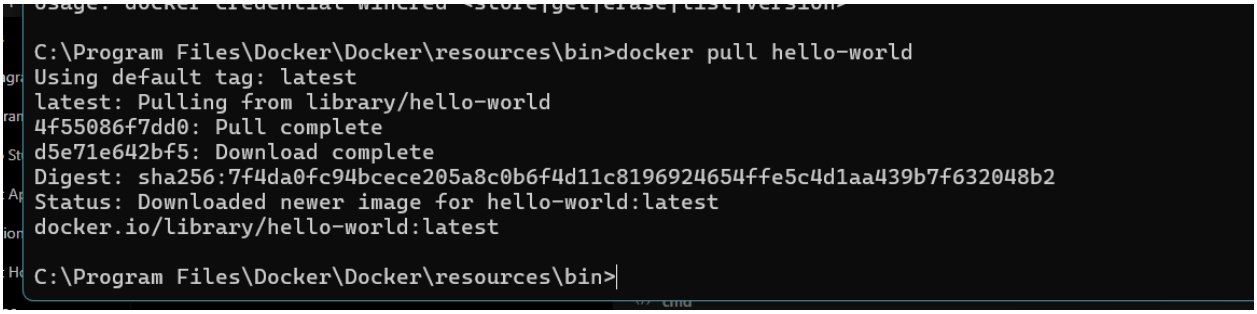
C:\Program Files\Docker\Docker\resources\bin>docker info
Client:
 Version:    29.2.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.18.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
 init version: de40ad0
 Security Options:
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.6.87.2-microsoft-standard-WSL2
 Operating System: Docker Desktop
 OSType: linux
 Architecture: x86_64
 CPUs: 8
 Total Memory: 7.43GiB
 Name: docker-desktop
 ID: 62ef7211-0656-47b9-bf58-2e050ae6ed2f
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 HTTP Proxy: http.docker.internal:3128
 HTTPS Proxy: http.docker.internal:3128
 No Proxy: hubproxy.docker.internal
 Labels:
  com.docker.desktop.address=npipe://\\.\pipe\docker_cli
 Experimental: false
 Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false
 Firewall Backend: iptables
```

Run an Existing Docker Image

STEP 1: Pull an existing image

docker pull hello-world

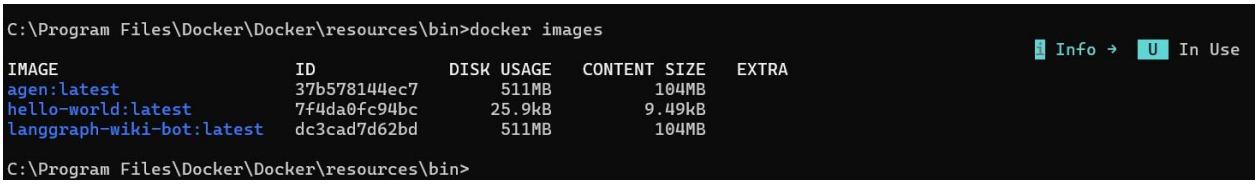
This downloads the **hello-world** image from Docker Hub.



STEP 2: Verify the downloaded images

docker images

This lists all the available images on your system.



STEP 3: Create and run a container from the image

docker run hello-world

- This command creates a new container and executes it.
- The message confirms Docker is installed and working correctly.

```
C:\Program Files\Docker\Docker\resources\bin>docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

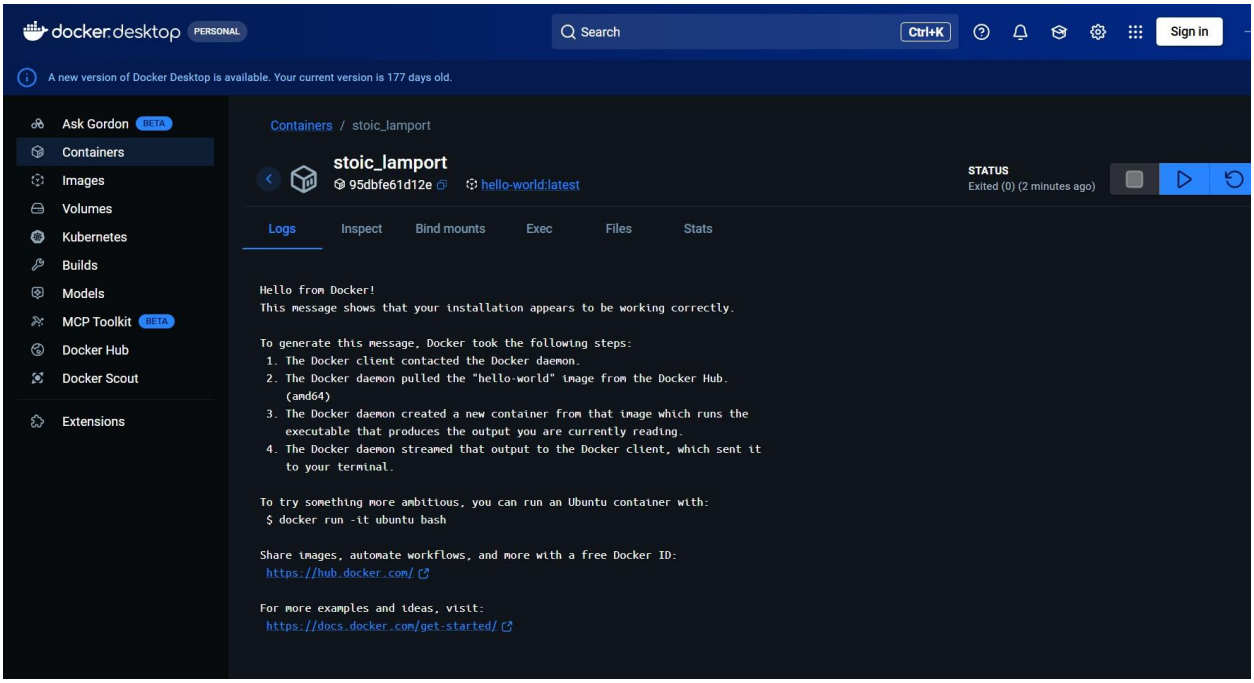
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/


C:\Program Files\Docker\Docker\resources\bin>
```



STEP 4: Verify Container Creation

docker ps –a

This shows all the containers, including running and stopped containers.

```
C:\Program Files\Docker\Docker\resources\bin>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Program Files\Docker\Docker\resources\bin>docker ps -a
CONTAINER ID   IMAGE               COMMAND          CREATED         STATUS                     PORTS     NAMES
4c63983887fd   hello-world:latest  "/hello"         2 minutes ago   Exited (0) 2 minutes ago             beautiful_boh
r
95dbfe61d12e   hello-world         "/hello"         5 minutes ago   Exited (0) 2 minutes ago             stoic_lamport
b34bdeb94220   7713943a2e5c        "python app.py"  5 months ago    Exited (1) 5 months ago              loving_mestor
f
e388f4127e55   7713943a2e5c        "python app.py"  5 months ago    Exited (1) 5 months ago              zen_bose
488bb7ba6080   7713943a2e5c        "python app.py"  5 months ago    Exited (1) 5 months ago              affectionate_
yalow
602362d480ba   7713943a2e5c        "python app.py"  5 months ago    Exited (1) 5 months ago              cool_wiles
```

STEP 5: Lifecycle of Docker Execution

Image → Run → Create → Container → Execute → Output

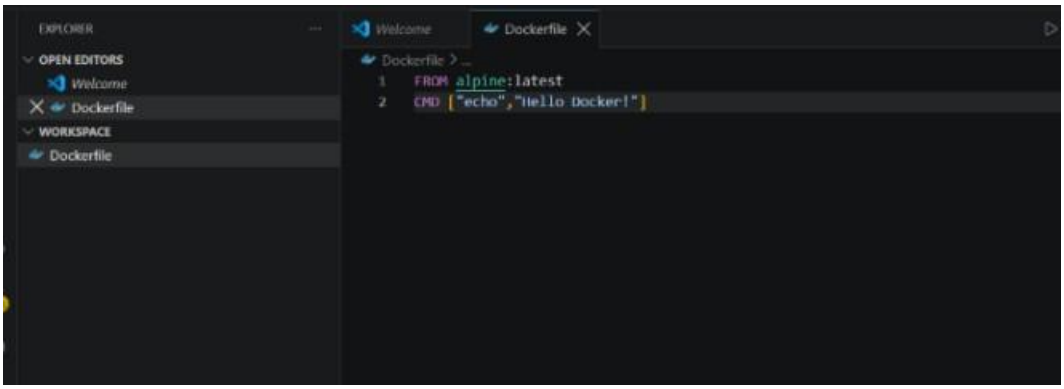
<u>Creating Own Image</u>

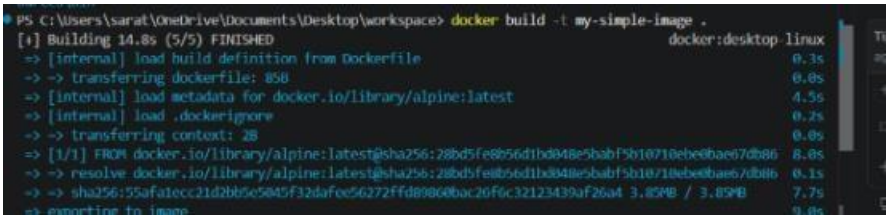
Create Docker File:

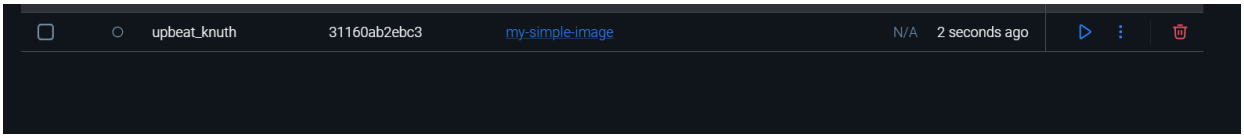
FROM alpine:latest

CMD ["echo", "Hello from my custom image"]

Docker Desktop shows local images confirming the image was built and stored locally.







Run Node.js in Docker

Step 1: Pull the Node.js image

docker pull node

This downloads the latest Node.js image from Docker Hub.

```
C:\Program Files\Docker\Docker\resources\bin>docker pull node
Using default tag: latest
latest: Pulling from library/node
7377a6b98a02: Pull complete
1da3cb2f93f2: Pull complete
68b64c51cda3: Pull complete
d74736caedd4: Pull complete
ec935196e6a0: Pull complete
e7e80b46ea4e: Pull complete
755d4732e515: Pull complete
822156e9d942: Download complete
1bbc6a8895d4: Download complete
Digest: sha256:bde0dae02f2b12d2bce5ee72b2432f0e511767b7b2dc4dd3b064df11ae422fee
Status: Downloaded newer image for node:latest
docker.io/library/node:latest
```

Step 2: Run a container from the Node.js image

docker run node

Creats and runs a new Node.js container.

```
C:\Program Files\Docker\Docker\resources\bin>docker run node
```

Step 3: Start an existing container

docker start <container_name>

```
C:\Program Files\Docker\Docker\resources\bin>docker start admiring_lamport
admiring_lamport
```

Step 4: Run Node.js in interactive mode

docker run -it node

```
C:\Program Files\Docker\Docker\resources\bin>docker run -it node
Welcome to Node.js v26.7.0.
Type ".help" for more information.
> let a=1;
undefined
> let b=2;
undefined
> console.out(a+b);
Uncaught TypeError: console.out is not a function
> console.log(a+b);
3
undefined
>
```

Step 5: Stop the running container

docker stop <container_id>

```
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\Docker\Docker\resources\bin>docker stop 15f98f8f5df177c58cf63d5a776f719da4d5bb40bc19c986543114c81290cc2
5
15f98f8f5df177c58cf63d5a776f719da4d5bb40bc19c986543114c81290cc25

C:\Program Files\Docker\Docker\resources\bin>
```

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result:

The Docker setup was verified by successfully running containers, including Node.js. This confirmed proper container execution and the ability to create and manage custom images.